CETP: from capstone to research + product

Written 2026-10-12, after auditing the repo against the LLD/deck claims. Every number below was re-run from the actual scripts, not taken from a report.

---

0. Verified state of the repo (so the rest of this is grounded)

Thing	Verified finding
Headline 21.60% vs 23.03%	**Reproduces exactly** (`addition4_reevaluate.py` , multi-thread compute block)
Bootstrap CI [0.30, 2.55]	Present in `results/addition4_compute_benchmark_report.md` , consistent
Parallelism mechanism	**Confirmed**: r=0.458, p=0.037 (`results/fix3_parallelism_mechanism_summary.csv`)
Machine count	**5** — c5a, c7i, m5a, r5n, z1d. The deck's "7" is wrong.
Classifier macro-F1	**0.375 on corrected data**; 0.562 is the *pre-buffer-fix* number. io-class recall = **0.0**.
CI/CD gate	**Exists and runs** — `gate/cetp_gate.py` , 295 lines, PASS/BLOCK/WARN, GitHub workflow present
Self-calibration loop	**Exists** — `measurement_store.py` (SQLite, 3 tables), `addition7_online_loop.py`
13 features	Match the claimed list exactly, no `prod_*` leakage in the list

The one serious problem found

9 of 14 analysis scripts still read `data/raw/tpch_dataset.csv` — the dataset with the known-buggy buffer counts. Including `addition4_reevaluate.py` (the headline), `phase7_final_evaluation.py` , `confidence_abstention.py` , `learning_curve.py` , `phase6_scaling_regressor.py` , `online_predictor.py` , `pre_pipeline_validation.py` .

I tested whether the headline survives. **It does, and improves slightly:**

```
corrected dataset, multi-thread compute, leave-one-machine-out MAPE
  naive (=1.0)                      31.97
  naive-linear (1/bandwidth_ratio)  23.03
  analytical roofline               21.50   <- was 21.60 on buggy data
  bottleneck-gated                  22.54
  learned quantile (p50)            26.20
```

So the science holds. But until every script reads one canonical dataset, any examiner or reviewer who greps for `data/raw` has a free shot at the whole thing.

Second problem: the repo tells two contradictory stories

- `README.md` : the analytical roofline beats the baseline significantly. **(current, true)**

- `results/phase7_report.md`: "a single hardware ratio is the best-performing estimator and **no** more sophisticated approach improves on it." **(true before the multi-core fix, now stale)**

Both were true in their own epoch. Side by side in one repo they read as contradiction.

---

1. Credibility fixes — do these first, they take a day

1. **One canonical dataset.** Point every script at `data/corrected/tpch_dataset_corrected_v2.csv`. Delete or clearly quarantine `data/raw/`.
2. **One canonical results script.** `make results` regenerates *one* table that every document quotes. Add a header to `phase7_report.md`: "SUPERSEDED by Addition 4 — the compute signal used here is the single-core benchmark, since replaced."
3. **Fix the F1 number everywhere to 0.375**, and state that io recall is 0.0. The LLD's 0.562 is the pre-correction figure. Note that structural-only features score **0.454**, better than both the 0.375 buffer baseline and the 0.336 combined model.
4. **Machine count is 5.** Fix the deck.
5. **Nested-CV leakage audit.** `p_compute/p_bandwidth/p_io/p_mixed` and `analytical_scaling` are features of the learned model. Confirm that in each leave-one-machine-out fold the classifier producing those probabilities was refit *without* the held-out machine. If it was fit once on everything, the learned model's numbers are optimistic and must be redone.
6. **Report q-error alongside MAPE.** `max(pred/actual, actual/pred)` is the metric the DB community expects. MAPE alone will be flagged by any SIGMOD/VLDB reviewer.

---

2. Make the results actually better — in leverage order

2A. Kill the hand-assigned bottleneck labels. Measure resource elasticity instead.

This is the single biggest scientific upgrade available, and it is cheap.

Right now ground truth is a 21-entry Python dict of your own judgement calls (`BOTTLENECK_LABELS`). The "classifier" is learning to reproduce your opinions from 21 examples, and those opinions become the weights in the analytical formula. Nothing about that is defensible in review, and Q2/Q13 being acknowledged judgement calls proves the point.

Replace it with a **measured** quantity. For each query, on one machine, sweep each resource and measure the elasticity of runtime:

- **cores**: `max_parallel_workers_per_gather` = 0,1,2,4,8 (and `taskset` to pin core count)
- **memory bandwidth**: run a bandwidth antagonist co-process (`stress-ng --vm`, or a STREAM loop pinned to other cores) at several intensities
- **cache / buffer residency**: vary `shared_buffers`, and cold vs. warm page cache

Fit `d log(runtime) / d log(resource)` per resource. You get a continuous 3-vector per query: *measured sensitivity*, not a guessed class. Benefits, all of them real:

- the classifier becomes a **regression onto measured targets** — no more 4-way opinion labels, no more macro-F1 0.375 with io recall 0.0
- the analytical formula's weights become **measured elasticities** instead of softmax outputs, which turns a heuristic into a derivation
- you can finally explain *why* the roofline formula works, which is what makes it publishable
- it scales: 200 queries is 200 measured vectors, no human labelling bottleneck

Cost: a few hours of machine time on one box. Highest ROI item on this list.

2B. Ablate the classifier before you trust it

Run the analytical formula with: (a) learned `p_*` — current, (b) one-hot hand labels, (c) **uniform 0.25 weights**, (d) measured elasticities from 2A.

My prediction: **(a) ≈ (c)**, i.e. the classifier is decorative and the compute/bandwidth ratios are doing all the work. Evidence for this is already in your own data — the classifier's macro-F1 falls from 0.562 to 0.375 on corrected data while the analytical MAPE barely moves (21.60 → 21.50). If (a) ≈ (c) holds, say so plainly; a correctly-identified decorative component is a finding, and 2A is then the fix rather than a nice-to-have.

2C. Replace quantile GBR with conformal prediction

Your coverage is broken and you know it: p95 covers 80.7% (nominal 95%), p99 covers 94.4% (nominal 99%). Gradient-boosted quantile regression offers **no coverage guarantee at all**, so "p95" currently means nothing checkable.

Use **conformalized quantile regression** (Romano et al. 2019) with **group/cluster conformal calibration at the machine level** — matching your leave-one-machine-out structure. For the held-out-machine case, the correct tool is **weighted conformal prediction under covariate shift** (Tibshirani et al. 2019), with weights derived from your existing hardware-distance metric. That is *exactly* your problem, and it converts your abstention threshold from a hand-tuned 0.6 into a principled quantity.

Payoff: you stop saying "our p95 under-covers" and start saying "we guarantee ≥95% coverage, and here is the interval width that costs." For a product that gates deployments, a guarantee is the entire value proposition.

2D. Use the plan tree, not just the root node

Six scalars from the root node throws away almost everything. The plan is a tree with node types, join methods, estimated vs. actual rows, loop counts, and per-node buffer activity.

You already have evidence this matters: structural-only features scored **0.454** macro-F1 vs 0.375 for buffers. `extract_plan_features.py` and `results/plan_structure_features.csv` exist. The reason 13 combined features *hurt* (0.336) is 21 training rows, not uninformative structure — your own `fix2` verdict says this.

So: more queries (§3) first, then a plan encoder. Simple version is per-node-type aggregated cost/rows/buffers (~40 features). Full version is a tree-structured net in the QPPNet lineage. Don't attempt a GNN at n=21.

2E. Model in log space, and fix the loss

Scaling factors are multiplicative. Fit `log(prod/dev)` with squared loss — that is geometric-mean-optimal and aligned with the ratio-error metrics you actually report. Current setup optimizes squared error on the raw ratio, which over-weights the slow tail asymmetrically.

2F. Make the dev side a real dev machine

Every pair today is EC2→EC2. The product claim is "predict prod from a developer's laptop." Add your own Mac (arm64, M-series) as a dev reference. That is a much harder transfer — different ISA, different memory system, different core topology — and the actual use case. If the method survives arm64-laptop → x86-EC2, the claim becomes far stronger. If it doesn't, you have found the real boundary of the method, which is equally worth reporting.

---

3. Dataset scale — this is the actual blocker

Every limitation in `phase7_report.md` reduces to "5 machines, 21 queries, SF1, EBS-only, no concurrency." No amount of modelling cleverness fixes that. Nothing else on this list matters as much.

The good news: this is cheap. 21 queries × 20 reps at SF1 is minutes of machine time. On spot pricing, 30 instances is **single-digit dollars**. The blocker is automation, not money and not access. A Terraform/Ansible (or packer AMI + boto3 driver) one-command harness is a weekend of work and it unlocks everything else.

Machines: 5 → 30+, covering axes deliberately

Don't add machines randomly; cover the dimensions that let you make claims:

- **ISA**: Intel x86, AMD x86, **Graviton arm64** (c7g/m7g/r7g). arm64 is the interesting one — it is where "hardware signature" stops being a 2-scalar story.
- **core count**: 2 / 4 / 8 / 16 — you have *already shown* aggregate cores drive the effect, so this axis is now theory-driven, not fishing.
- **storage**: gp3, io2, and **instance-local NVMe** (i4i, i3en). This is non-optional. Right now all 5 machines are EBS under one AMI, which means your `io` bottleneck class reflects buffer-touch patterns under uniform storage, **not storage diversity** — your own limitations section concedes this. Until you add NVMe, the io class is not a real class.
- **one bare-metal** instance, to separate virtualization overhead from hardware.
- **one laptop** (see 2F).

Queries: 21 → ~230

- TPC-H (22, keep Q15 this time — it needs a view/CTE, which is a 30-minute fix, not a reason to drop a query and then explain the gap in every document)
- **TPC-DS (99)** — far richer plan diversity
- **Join Order Benchmark (113)** — the standard in learned-query-optimization papers. Including it makes your work legible to the community you want to publish in.
- ClickBench, optionally, for a different query shape

At ~230 queries the hand-labelling problem dies on its own, the plan encoder becomes trainable, and leave-one-query-out stops being a 21-sample exercise.

Second and third axes

- **Scale factor**: SF1 / SF10 / SF100. Generalization across *data size* is a second axis and a result in its own right: does cross-hardware transfer hold as data grows? Your SF10 validation was skipped; this is where it comes back.
- **Concurrency**: everything today is one query at a time. Production is concurrent. Even two levels (1 client, 8 clients) makes CETP credible as a *production* predictor and opens a genuinely understudied interaction between contention and hardware transfer.

Publish the dataset

30 machines × 230 queries × 3 scale factors × 2 concurrency levels, with plans, hardware signatures, and elasticity profiles, is a **citable artifact** and a legitimate benchmark/datasets-track submission on its own. Nothing like it is public. Even if every model you try loses to naive-linear, the dataset is a contribution.

---

4. The novel research contributions — pick one headline, not five

N1. Plan drift: the planner is not hardware-invariant ← strongest, do this one

Your entire method rests on an assumption that is **false**: that workload cost is hardware-invariant and only machine capability changes. In PostgreSQL the *plan itself* changes across hardware — `max_parallel_workers_per_gather` depends on core count, and `effective_cache_size`, `work_mem`, `shared_buffers`, `random_page_cost` are all set from hardware. Different machine, different plan. Your dev-side cost vector describes a plan that prod will not run.

You have already stumbled onto this: the whole multi-core finding is *because* the planner chose parallel plans with different worker counts, and `fix3` confirmed it causally.

The research question nobody has answered: how much of cross-environment prediction error is **plan divergence** versus **capability scaling** — and can plan divergence be predicted from the dev plan plus the hardware signature alone?

The experiment that answers it: run each query on prod twice — once with prod's own plan, once with the dev plan *forced* (via `pg_hint_plan`, or by pinning the relevant GUCs to dev's values). The difference isolates plan drift. Then decompose total error into the two terms.

This is novel, mechanistically interesting, directly actionable (if drift dominates, the fix is to predict the plan, not the runtime), and it reframes the whole project from "an ML regression on 5 machines" into "a measurement study of why cross-environment prediction is hard." The latter is publishable at n=30 in a way the former is not.

Working title: *Plan Drift: Separating Planner Divergence from Hardware Scaling in Cross-Environment Query Runtime Prediction.*

N2. Resource elasticity as a learnable query descriptor

§2A elevated to a contribution. Measured sensitivity vectors replace opinion labels; they are predictable from the plan; they become the weights in a physics-grounded scaling law. Clean, well-posed, and it is what makes your roofline formula principled rather than heuristic. This is the *enabling method* for N1 and N4.

N3. Budgeted cold start: calibrate a new machine in 3 runs, not 21

Your `addition7` loop reveals 12 queries in arbitrary order and gets 77.7% → 15.0%. Make the selection principled: choose the k queries that maximally reduce uncertainty about the new machine (D-optimal design, max mutual information, or diversity in elasticity space). Evaluate error-vs-k for random / coverage-greedy / your acquisition function.

Cheap — **you can run this on the data you already have** — highly product-relevant, and it makes the self-calibration loop a contribution instead of a demo.

N4. Distribution-free SLA gating with coverage guarantees

§2C elevated. Conformal risk control for a CI/CD decision: given "no more than 5% of approved deployments may breach SLA," *derive* the gate threshold with a guarantee instead of tuning it. Your risk-coverage curve against random abstention is already the right framework; conformal makes it rigorous. Pairs naturally with selective-prediction theory.

N5. Learned hardware embeddings (needs 30+ machines)

Factorize the machine×query runtime matrix into latent hardware vectors and query weight vectors, then learn a probe→embedding regression so a new machine is placed from two cheap microbenchmarks. Answers "are two scalars enough?" — which you currently assume rather than test.

Recommended story: N2 enables, N1 is the headline, N4 + N3 are the system. One coherent paper: *measure what queries are sensitive to, separate planner drift from capability scaling, gate with guarantees, cold-start cheaply.*

---

5. Prior work you must read, cite, and baseline against

Not optional. A reviewer who knows this literature will reject in one line if it is missing.

- **Hilprecht & Binnig, "Zero-Shot Cost Models for Out-of-the-box Learned Cost Prediction," VLDB 2022** — the closest prior work: learned cost models that transfer to unseen databases, plus follow-on work on transfer across hardware/DBMS. **Read this first.** Either baseline against it or state precisely why your setting differs.
- **Marcus & Papaemmanouil, "Plan-Structured Deep Neural Network Models for Query Performance Prediction" (QPPNet), VLDB 2019** — the plan-tree encoder lineage for §2D.
- **Marcus et al., Neo (VLDB 2019) and Bao (SIGMOD 2021)** — learned-optimization context.
- **Ganapathi et al., "Predicting Multiple Metrics for Queries," ICDE 2009** — the classic query-performance-prediction paper.

- **Williams, Waterman & Patterson, "Roofline," CACM 2009** — your analytical estimator *is* a roofline model. Cite it as such; it immediately legitimizes the formula.
- **Romano, Patterson & Candès, "Conformalized Quantile Regression," NeurIPS 2019**; **Tibshirani et al., "Conformal Prediction Under Covariate Shift," NeurIPS 2019**; Angelopoulos & Bates, conformal prediction tutorial / conformal risk control — for §2C and N4.

(Verify exact venues and years before citing — these are from memory.)

Your honest differentiator: zero-shot cost models transfer across *databases* and predict *mean runtime*. CETP transfers across *hardware*, predicts *percentiles for an SLA decision*, and *abstains when it does not know*. That is a real gap. But you have to say it explicitly, in the intro, with the citation — not leave it for a reviewer to notice.

---

6. Product

The one reframe that matters: sell regression detection, not absolute prediction

21.5% MAPE will never gate a 500ms SLA. Your own Phase 7 numbers prove it: **54.5% of BLOCK verdicts were false alarms**, and the PASS zone held 4 of 840 pairs. As an absolute predictor feeding a hard gate, this is not shippable, and no amount of modelling at n=5 will change that.

But **"this PR makes q7 2.3× slower on prod-class hardware"** is:

- **easier** — comparing two versions of a query on the *same* hardware pair cancels much of the hardware term and most of the systematic bias
- **more useful** — regressions are what CI is for; nobody merges a PR hoping for an absolute latency number
- **defensible today**, at your current accuracy, without more data

Ship regression detection first. Keep absolute p95 prediction as the research track.

Positioning

"Performance CI for Postgres." Pre-merge, not post-incident. The adjacent tools — pganalyze, Datadog DBM, pgMustard, Metis, EverSQL — are all post-hoc observability or tuning. Nobody predicts production latency from a dev box before the code merges. That slot is open.

The moat is the hardware signature registry

Every user who runs the microbenchmark on their machine contributes a signature. The registry shrinks everyone's cold start. That is a genuine network effect and it is the actual business — not the model. Your SQLite `measurement_store.py` is already the right primitive; it needs a server mode and a sharing protocol.

Engineering to ship

- `pip install cetp` and a **GitHub Action** — the gate already works, it needs packaging

- real ingestion from production: `auto_explain` / `pg_stat_statements` → the `measurements` table (right now the loop is seeded from a CSV)
- **plan redaction** — EXPLAIN output contains query literals, which means customer PII. Strip before anything leaves the user's machine. Do this before the first external user, not after.
- Postgres 13–17 coverage; plan JSON format drifts between versions
- per-repo calibration state, and multi-tenancy in the store

Market the abstention

"We tell you when we don't know, and we say run it for real" is not a weakness to hide — it is the single most trustworthy thing about the system and the reason someone would put it in a deployment path. Your risk-coverage curve is the sales slide.

Later expansion

MySQL, DuckDB, and — the biggest commercial wedge — **cloud warehouse sizing**: "which Snowflake warehouse size does this query need?" That is the same problem with a dollar figure attached to every wrong answer.

---

7. Twelve-week plan

Weeks	Work
1–2	§1 credibility fixes. One dataset, one results script, q-error, leakage audit. Read the §5 prior work and write the related-work section — it will change your framing.
3–4	Automated collection harness (Terraform/boto3, one command, N machines). Elasticity measurement rig (§2A) on one box.
5–7	Collect: 30 machine configs × (TPC-H + TPC-DS + JOB) × SF1/SF10, 2 concurrency levels. Include arm64, NVMe, bare-metal, laptop.
8–9	Elasticity labels replace hand labels. Plan-tree features. Conformal gate with group calibration.
10	**N1 plan-drift experiment**: forced-dev-plan vs native-plan on prod, error decomposition.
11	**N3 budgeted cold start**: acquisition function, error-vs-k curves.
12	Write-up, dataset release, arXiv.

Venues

- **aiDM** or **DBTest** (both co-located with SIGMOD) — the natural home for a rigorous empirical study with a public dataset. Realistic target.
- **CIDR** — if you lead with the N1 plan-drift framing as a vision paper.
- **VLDB demo** for the gate, **datasets/benchmark track** for the dataset.
- **ICDE short**.
- arXiv first, regardless.

---

8. The honest bar, and the one risk

What you have is a well-executed capstone with unusually honest reporting — the negative results in `phase7_report.md` are better science than most student projects produce, and the buffer-bug correction report is the kind of thing that builds trust. The confirmed parallelism mechanism is a real finding.

What it is not yet: enough data to support any claim about generalization. 5 machines, 21 queries, one scale factor, one storage type, no concurrency. Every reviewer will start there, and they will be right.

The risk: at 30 machines, naive-linear might *still* win.

Pre-register that outcome as a result, so you cannot lose. "On a 30-machine, 230-query, 3-scale-factor public dataset, a single memory-bandwidth ratio matches or beats learned cross-hardware cost models; here is the measurement study explaining why, and here is the dataset" is a genuinely useful paper — DBTest and aiDM publish exactly this, and it is worth more to the field than a fragile positive result at n=5. Decide now that you will report whichever way it lands, and the project becomes safe to scale up.